Enhancing Deadline Miss Early Detection for DAG Tasks with Variable Probabilistic Thresholds

HAYATE TOBA, Graduate School of Science and Engineering, Saitama University, Saitama, Japan
SANJOY BARUAH, Washington University in St Louis, St. Louis, United States
TAKUYA AZUMI, Saitama University Graduate School of Science and Engineering, Saitama, Japan

Autonomous driving systems require rigorous real-time performance to ensure safety. The entire process, from sensor data acquisition to vehicle control command generation, must meet strict end-to-end deadlines. If a deadline miss occurs, the system must promptly transition to a fail-safe state. To enhance safety, this paper proposes a method for early detection of deadline misses. This method models the autonomous driving system as a directed acyclic graph (DAG) comprising timer-driven and event-driven nodes, assigning dynamic time constraints to each node based on the end-to-end deadline and remaining workload. Conventional approaches depend on worst-case execution time (WCET) assumptions, which often lead to overly pessimistic deadline miss detection by neglecting the variability in execution times. In contrast, the proposed method incorporates probabilistic execution times to model variability, dynamically adjusting thresholds based on workload. By progressively tightening thresholds for jobs closer to the exit node, the method balances detection sensitivity with false alarm reduction. Experimental results demonstrate that our probabilistic method not only reduces the pessimism of the conventional WCET-based early-detection approach, which corresponds to the current state-of-the-art configuration when the probability threshold is set to 1.0, but also generalizes this baseline by incorporating execution-time variability and workload-aware threshold adaptation.

CCS Concepts: • **Computer systems organization → Real-time systems**.

Additional Key Words and Phrases: Autonomous driving system, probabilistic execution time, deadline miss early detection, DAG, event-driven task, timer-driven task

1 Introduction

Autonomous vehicles have emerged as a central focus of technological and societal advancements due to their transformative potential in addressing critical challenges, such as traffic congestion, accidents, rural transportation deficiencies, and labor shortages in logistics. Prominent autonomous driving platforms, including *Waymo Driver* [16], *Autoware* [6], and *Apollo* [1], continue to advance toward achieving full autonomy. The Society of Automotive Engineers categorizes autonomous driving into six levels, from zero to five [12], with higher levels indicating greater system responsibility. At level five, systems are required to ensure safety even in emergencies, fully eliminating human intervention.

To guarantee operational safety, autonomous systems must adhere to stringent real-time requirements. Even minor processing delays can result in catastrophic outcomes, such as accidents caused by delayed emergency braking or collision avoidance. Consequently, all processing stages, from sensor data acquisition to vehicle control

This article is an extended version of a paper presented at the 36th Euromicro Conference on Real-Time Systems (ECRTS 2024), and has been submitted to the Special Issue on Industrial Real-Time Systems.
This work was supported by JST FOREST Grant Number JPMJFR242G and JST PRESTO Grant Number JPMJPR21P1.
Authors' Contact Information: Hayate Toba, Graduate School of Science and Engineering, Saitama University, Saitama, Japan; e-mail: bm8183.tbtb@gmail.com; Sanjoy Baruah, Washington University in St Louis, St. Louis, Missouri, United States; e-mail: baruah@cs.unc.edu; Takuya Azumi, Saitama University Graduate School of Science and Engineering, Saitama, Japan; e-mail: takuya@mail.saitama-u.ac.jp.



This work is licensed under a Creative Commons Attribution 4.0 International License.
© 2026 Copyright held by the owner/author(s).
ACM 1558-3465/2026/4-ART
https://doi.org/10.1145/3808694

Just Accepted

command execution, must complete within predefined deadlines. Directed acyclic graphs (DAGs), annotated with end-to-end deadlines, are widely used to model these systems [4, 7, 10]. This representation captures intricate task dependencies and facilitates rigorous real-time analysis.

Traditionally, deadline miss detection in DAG-based systems occurs only at the end of processing flows. While effective in ensuring safety, this approach delays the identification of potential deadline violations, limiting the ability of the system to implement preemptive safety measures. To address this limitation, prior research introduced early detection method [17], which calculates *laxity* from end-to-end deadlines. This method enables transitions to safety modes, such as the Minimum Risk Maneuver (MRM), earlier in the processing sequence, thereby mitigating risks more effectively.

Despite its innovative approach, this early detection method assumes that all tasks execute at their worst-case execution time (WCET). In practice, the actual likelihood of tasks reaching their WCET is minimal, leading to overly pessimistic predictions. This conservatism results in premature deadline miss detections, triggering unnecessary safety transitions and reducing overall system efficiency. Building on this foundation, subsequent work [14] proposed a probabilistic model that treats task execution times as random variables. This approach introduced more realistic intermediate time constraints, thereby reducing the pessimism inherent in WCET-based methods. The core idea of this method is to signal a potential deadline miss when the probability of remaining execution time exceeding the *laxity* surpasses a predefined threshold. However, the use of a single probabilistic threshold for the entire DAG limits its adaptability to diverse task scenarios and execution conditions. As the WCET-based early detection method [17] represents the state-of-the-art for mixed timer-driven and event-driven DAGs, our probabilistic threshold mechanism can be viewed as a principled generalization of this deterministic baseline.

To address this limitation, this paper extends the probabilistic early detection method by introducing a workload-based threshold mechanism. Unlike previous methods , this approach dynamically assigns thresholds based on the cumulative workload of subsequent tasks. For this calculation, we consider two cases: one considers all tasks equally, while the other weights tasks based on their average execution times. This refinement acknowledges the interplay between preceding and following tasks. For tasks at the start of the DAG, the lowest threshold is assigned, while the highest threshold is assigned to terminal tasks. Intermediate tasks receive thresholds that increase proportionally to the remaining workload. This improvement takes into account the interaction between preceding and subsequent tasks. In particular, for tasks with substantial remaining workload, thresholds are relaxed to mitigate temporary deadline miss risks through favorable execution time variations, thereby reducing false detections while maintaining system reliability. This tailored approach reduces false detections while maintaining system reliability.

Implementation of this approach involves utilizing convolution operations to calculate intermediate time constraints based on task execution time distributions. As a secondary contribution, adopting circular convolution [11] in these calculations reduces computational time, further enhancing the computing efficiency. However, the workload-based threshold calculations do not rely on convolution, as the proposed algorithm inherently avoids its use in favor of simpler computations based on workload properties. This separation ensures computational efficiency for both components of the system, balancing detection accuracy and operational feasibility. These refinements contribute to improving the applicability of early detection methods for real-world autonomous systems.

The main contributions of this paper are summarized as follows:

- Formulating a workload-based threshold adjustment mechanism to enhance early detection accuracy by considering the remaining workload after each task.
- Proposing two cases for workload assignment: uniform assignment and average execution time-based assignment, with thresholds adjusted accordingly.

Just Accepted

- Conducting extensive experimental evaluations to validate the improvements in detection precision and computational efficiency compared to conventional methods.

The remainder of this paper is structured as follows. Section 2 outlines the system model. Section 3 specifies the problem model. Section 4 details the proposed approach. Section 5 presents the evaluation results. Section 6 reviews related work. Section 7 concludes with future research directions.

2 System Model

This section presents the system model used in this paper, including a description of the DAG model and the definition of probabilistic execution time.

2.1 DAG-based Workload Model

An autonomous driving system is represented as a directed acyclic graph (DAG), where nodes represent tasks and directed edges represent data flow between tasks. Sensor data, including input from cameras, LiDAR, and the global navigation satellite system (GNSS), are obtained with different periods. Using these inputs, the system performs operations such as self-localization, object perception, and route planning, ultimately producing a single vehicle control command.

The DAG model described in this paper has multiple entry nodes with different periods and a single exit node. A deadline is assigned to the exit node, which outputs the vehicle control command, to ensure that the system meets real-time constraints. A simple example of such a DAG is illustrated in Fig. 1.

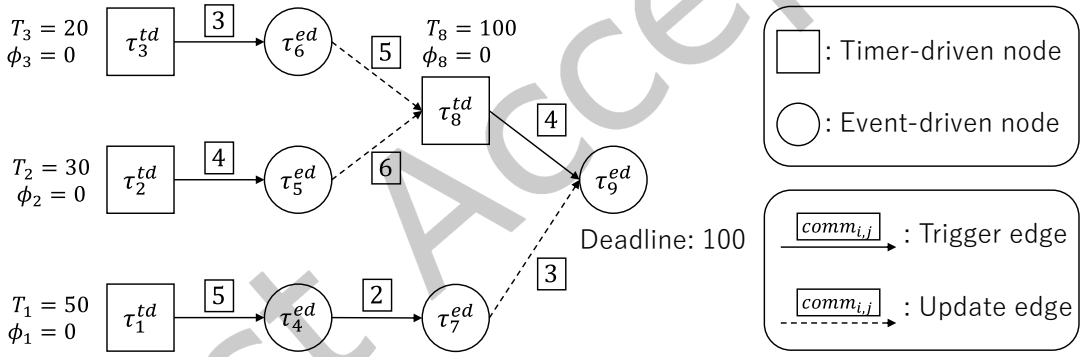


Fig. 1. Example of a simple DAG.

DAG-related notations used in this paper are listed in Table 1. A DAG is represented as $G = (V, E)$, where V is the set of all nodes ($V = \{\tau_1, \tau_2, \ldots, \tau_n\}$), $n = |V|$ is the total number of nodes, and E is the set of all edges. Each node τ_i represents a task, and edges $e_{i,j} \in E$ represent data communication between tasks.

Nodes in a DAG are classified as follows:

Timer-driven nodes **(τ_i^{td})** These nodes are triggered periodically with a period T_i and an offset ϕ_i, indicating the start time of the first execution.

Event-driven nodes **(τ_i^{ed})** These nodes are triggered when data from their predecessor nodes arrive.

Each execution of a node is referred to as a job, and the kth job of a node τ_i is denoted by $\langle \tau_i, k \rangle$. The execution time of a node τ_i is probabilistic and is represented by the random variable (RV) X_i, with its probability distribution denoted as $\mathbb{X}_i$.

Edges in the DAG are classified into two types:

Just Accepted

Table 1. DAG notations

Symbol	Description
n	Total number of nodes in a DAG
τ_i	Node
τ_i^{td}	*Timer-driven node*
τ_i^{ed}	*Event-driven node*
V^{td}	Set of all *timer-driven nodes* in a DAG
T_i	Period of τ_i^{td}
ϕ_i	Offset of τ_i^{td}
$\langle \tau_i, k \rangle$	k^{th} job of τ_i
$\langle \tau_i^{td}, k \rangle$	k^{th} job of τ_i^{td}
$\langle \tau_i^{ed}, k \rangle$	k^{th} job of τ_i^{ed}
X_i	Probabilistic execution time of τ_i
$\mathbb{X}_i$	Probability distribution of X_i
$e_{i,j}^{tr}$	*Trigger edge* from τ_i to τ_j
$e_{i,j}^{up}$	*Update edge* from τ_i to τ_j
$comm_{i,j}$	Worst-case communication time of $e_{i,j}$

Trigger edges **($e_{i,j}^{tr}$)** These edges indicate that event-driven successor nodes are triggered upon receiving data from predecessor nodes.

Update edges **($e_{i,j}^{up}$)** These edges indicate that data is stored temporarily in memory and accessed later when the successor node is triggered.

Each edge $e_{i,j}$ has a worst-case communication time $comm_{i,j}$. Although variations in communication time can exist, this paper assumes worst-case communication times to simplify the problem.

An example classification of nodes and edges in Fig. 1 is as follows:

- Timer-driven nodes: $\{\tau_1^{td}, \tau_2^{td}, \tau_3^{td}, \tau_8^{td}\}$.
- Event-driven nodes: $\{\tau_4^{ed}, \tau_5^{ed}, \tau_6^{ed}, \tau_7^{ed}, \tau_9^{ed}\}$.
- Trigger edges: $\{e_{1,4}^{tr}, e_{2,5}^{tr}, e_{3,6}^{tr}, e_{4,7}^{tr}, e_{8,9}^{tr}\}$.
- Update edges: $\{e_{5,8}^{up}, e_{6,8}^{up}, e_{7,9}^{up}\}$.

2.2 Probabilistic Execution Time

For a more realistic approach to early deadline miss detection, the execution time of each node τ_i is modeled in this paper as a random variable, denoted by X_i. The random nature of X_i is captured by its probability distribution, expressed as $\mathbb{X}_i = (x_i, p_i)$. In this representation, $x_i = x_{i,1}, x_{i,2}, \ldots, x_{i,l}$ is an ordered set partitioning the range of possible execution times for τ_i into $l-1$ distinct intervals. The corresponding probabilities, $p_i = p_{i,1}, p_{i,2}, \ldots, p_{i,l}$, represent the likelihood of the execution time of τ_i falling within each interval, where $p_{i,m} = P(x_{i,m-1} < X_i \leq x_{i,m})$. To clarify, $x_{i,0}$ is defined as 0, $x_{i,l}$ equals the WCET of τ_i, and the probabilities satisfy $\sum_{m=1}^{l} p_{i,m} = 1$. As an example, consider τ_1 in Fig. 2, where $P(0 < X_1 \leq 5) = 0.8$ and $P(5 < X_1 \leq 10) = 0.2$.

To ensure computational stability and reproducibility of the convolution operations, assumptions regarding the support and resolution of the probability distributions are imposed as follows:

Just Accepted

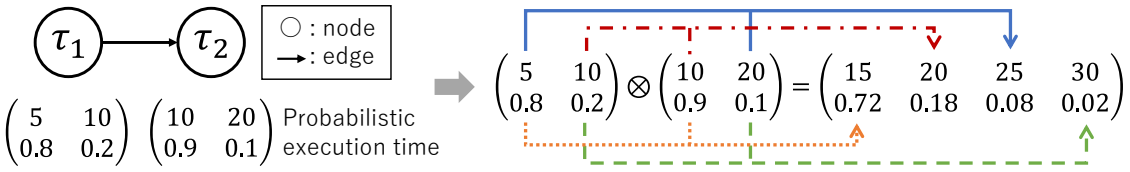


Fig. 2. Probabilistic total execution time of two nodes.

Bounded Support The execution time random variable X_i is defined over a finite range $[0, WCET_i]$, where $WCET_i$ is the worst-case execution time. Probabilities outside this range are zero. This bounded support ensures that the convolution size remains finite.

Discrete Resolution The time domain is discretized into fixed intervals (bins) of size Δt (time unit). All execution times, periods and deadlines are represented as integer multiples of Δt. A finer Δt provides higher accuracy but increases the computational cost, as analyzed in Section 5.5.

The probability distributions used in this study are derived from statistical analysis of execution data collected during the operation of an autonomous driving system. These distributions are designed to closely mirror real-world execution patterns. The probabilistic execution-time distributions used in this paper are derived from real-world *Autoware* [6] execution traces, as described in Section 5.1. While dependencies between task execution times might exist due to shared resources or inter-task interactions, for simplicity, this paper assumes statistical independence among them. This simplification facilitates the application of convolution operations to efficiently aggregate execution times.

We acknowledge that in the absence of such conservative modeling, assuming independence among positively correlated tasks could theoretically lead to an underestimation, resulting in optimistic deadline miss predictions. Regarding extensions to partially dependent distributions, approaches such as dependency bounds or Copulas could be applied. However, these methods often introduce significant computational complexity or overly pessimistic bounds. A practical future extension would be to model highly correlated sub-chains as single "super-nodes" to internalize dependencies before performing convolution, balancing accuracy and computational feasibility.

To determine whether the system adheres to its time constraints, it is critical to compute the cumulative execution time across a sequence of nodes. Unlike fixed execution times, probabilistic values cannot be summed directly. To address this, convolution is employed to model the aggregate execution time as a new random variable. For two nodes with probabilistic execution times X_i and X_j, their convolution, $X_i \otimes X_j$, is defined as:

$$P(Y = y) = \sum_{m=1}^{l} P(X_i = x_{i,m}) \cdot P(X_j = y - x_{i,m}), \tag{1}$$

where Y represents the total execution time as a random variable, and the assumption of independence between X_i and X_j holds. A simplified visualization of this process is provided in Fig. 2.

3 Problem Model

Autonomous driving systems must adhere to strict time constraints to ensure reliable and timely operation. To achieve this, it is essential to evaluate the dependencies among jobs and to establish how the start time of each job influences the likelihood of missing a deadline. However, performing timing analysis on a DAG that incorporates nodes triggered at varying periods presents a significant challenge [2].

Just Accepted

Table 2. Notations for the problem model

Symbol	Description
sg_i	Subgraph
$head(sg_i)$	Index of the head *timer-driven node* of sg_i
T_{sg_i}	Period of sg_i
$T_{sg_{exit}}$	Period of the subgraph that includes exit node
$j\tau_i$	*Join node*
$t\tau_i$	*Tail node*
$sub(t\tau_i)$	Index of the subgraph that contains $t\tau_i$
$pred^{tr}(\tau_i)$	Set of nodes with a *trigger edge* to τ_i
$DFC(t\tau_i)$	*Data freshness* constraint of the data generated by $t\tau_i$
$data(t\tau_i, k)$	Data generated by the *tail node* job $\langle t\tau_i, k\rangle$
$stamp[data(t\tau_i, k)]$	Timestamp of $data(t\tau_i, k)$
D	End-to-end deadline
d_k	Deadline of kth job of exit node
HP	Hyper-period of a DAG
JLD	Set of all the job-level dependencies in the analysis interval
$(\langle\tau_i, k\rangle \rightarrow \langle\tau_j, s\rangle)$	Job-level dependency from $\langle\tau_i, k\rangle$ to $\langle\tau_j, s\rangle$
N_{sg_i}	Number of times sg_i is executed during the analysis interval
$RST(\tau_i, k)$	Reference start time of $\langle\tau_i, k\rangle$
$RFT(\tau_i, k)$	Reference finish time of $\langle\tau_i, k\rangle$

To address this complexity, the method described in Section 3.1 segments the DAG into subgraphs, each driven by a single period. In these subgraphs, only the head node operates as a timer-driven node. We then define the necessary time constraints in Section 3.2 and identify the job-level dependencies following the methodology outlined in Ref. [17] in Section 3.3. A comprehensive summary of the notations used throughout this section is provided in Table 2.

3.1 Dividing the DAG into Subgraphs

When a DAG includes *timer-driven nodes* with varying periods, it can be segmented into distinct subgraphs, each characterized by a single period. Each subgraph, denoted as sg_i, contains one *timer-driven node* and potentially multiple *event-driven nodes* that are either directly or indirectly triggered by the *timer-driven node* (e.g., $sg_i = \tau_a^{td}, \tau_b^{ed}, \ldots, \tau_c^{ed}$).

The triggering sequence begins with the *timer-driven* node, which activates based on its specified period and offset, and continues through a series of consecutively triggered nodes. As a result, the period of a subgraph is equivalent to that of its head *timer-driven node*. The period of sg_i, denoted by T_{sg_i}, is defined as follows:

$$T_{sg_i} = T_{head(sg_i)}, \tag{2}$$

where $head(sg_i)$ refers to the index of the head *timer-driven node* within sg_i.

A practical example of this segmentation is shown in Fig. 3, which illustrates the division of the DAG from Fig. 1 into subgraphs. For instance, the subgraph sg_1 comprises one *timer-driven node* τ_1^{td} and two *event-driven nodes*, τ_4^{ed} and τ_7^{ed}. Consequently, sg_1 has a period of $T_{sg_1} = T_{head(sg_1)} = T_1 = 50$, as calculated using Equation (2).

To analyze job-level dependencies effectively, it is crucial to consider data transmission between subgraphs. Nodes involved in such data transfers are categorized as follows:

Just Accepted

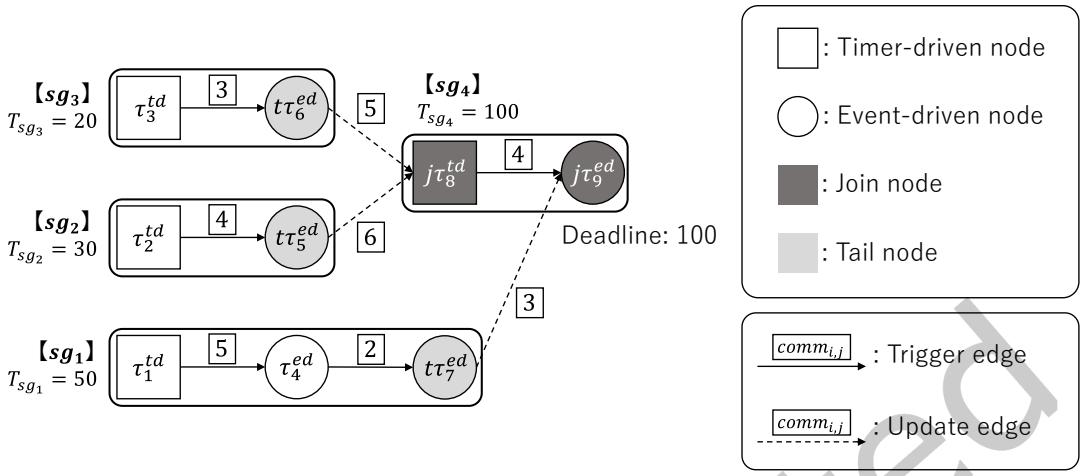


Fig. 3. Dividing the example DAG into a set of subgraphs.

Join nodes **($jt\tau_i$)** These nodes receive data from multiple subgraphs.
Tail nodes **($t\tau_i$)** These nodes send data to *join nodes* in other subgraphs.

For example, in Fig. 3, $jt\tau_8^{td}$ and $jt\tau_9^{ed}$ serve as *join nodes*, while $t\tau_5^{ed}$, $t\tau_6^{ed}$, and $t\tau_7^{ed}$ are *tail nodes*. It is important to note that a single node can function as both a *join node* and a *tail node*. For instance, if $jt\tau_9^{ed}$ has an outgoing edge to a different subgraph, it qualifies as both a *join node* and a *tail node*.

In the context of this paper, each node in the DAG is uniquely assigned to a single subgraph. Even when an *event-driven node* has predecessors in multiple subgraphs, it is triggered by predecessors within the same subgraph to which it belongs. Thus, any edge connecting a *tail node* to a node in another subgraph is classified as an *update edge*.

3.2 Definitions of Constraints

Autonomous driving systems must adhere to two critical constraints to ensure reliable and safe operation: (i) the *data freshness* constraint and (ii) the end-to-end deadline. This section provides detailed definitions for these constraints.

Since the environment surrounding an autonomous vehicle is constantly changing, using outdated data can significantly reduce the accuracy of operations such as self-localization and object perception [7, 15]. This reduction in accuracy poses a safety risk, making it imperative for each node in the system to process data that is sufficiently fresh. To address this, a *data freshness* constraint is introduced, which specifies the maximum permissible staleness of data used by each *join node*.

A *join node* can execute only when all its predecessor nodes provide data that complies with the *data freshness* constraint. Here, *data freshness* refers to the time elapsed from when a piece of data was timestamped to when it is utilized. Data originating from sensor nodes, which are the entry points of the DAG, are timestamped at the start of their execution. These timestamps remain unchanged as the data progresses through the system, until it is integrated at a *join node*. The *join nodes* update the timestamps of the data they generate, assigning their own execution start times as the new timestamps.

Just Accepted

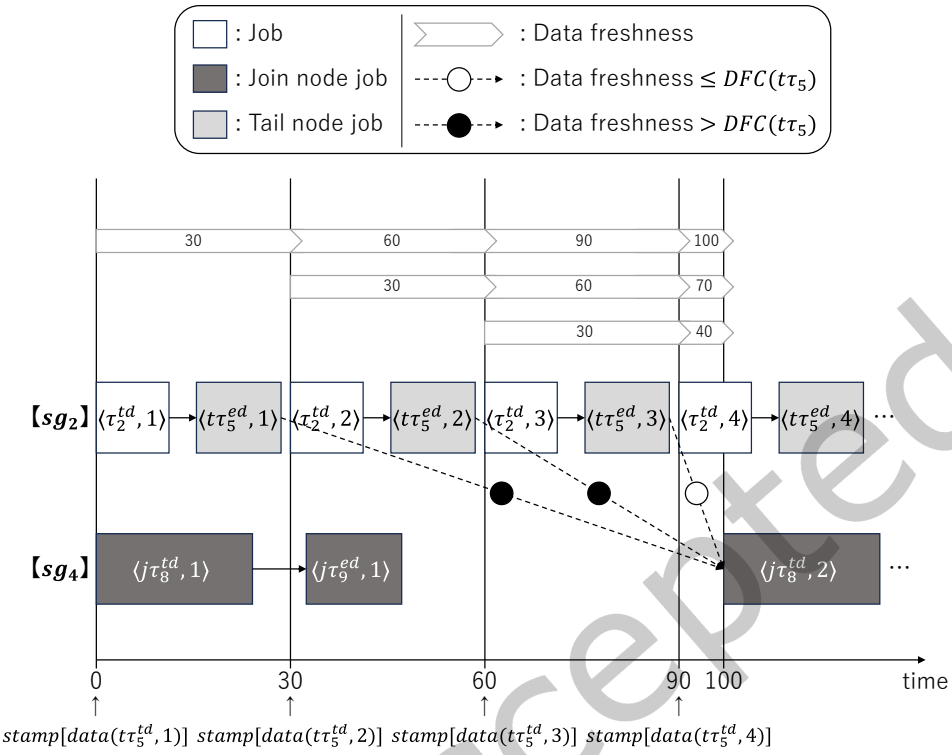


Fig. 4. The *data freshness* constraint for $t\tau_5^{ed}$ and $j\tau_8^{td}$.

The *data freshness* constraint for data generated by a *tail node* $t\tau_i$ is represented as $DFC(t\tau_i)$ and is defined as follows:

$$DFC(t\tau_i) = \alpha \times T_{sg_{sub(t\tau_i)}}, \tag{3}$$

where $\alpha \in \mathbb{R}^+$, and $T_{sg_{sub(t\tau_i)}}$ is the period of the subgraph that includes $t\tau_i$. As indicated in Equation (3), $DFC(t\tau_i)$ is a positive real multiple of $T_{sg_{sub(t\tau_i)}}$. The parameter α can be adjusted based on system requirements. A smaller α value enforces a stricter *data freshness* constraint, while a larger α value allows more leniency.

An example of the *data freshness* constraint for $t\tau_5^{ed}$ and $j\tau_8^{td}$ in Fig. 3, with $\alpha = 2$, is depicted in Fig. 4. In this example, data output from a *tail node* job $\langle t\tau_i, k\rangle$ is denoted as $data(t\tau_i, k)$, with its timestamp represented as $stamp[data(t\tau_i, k)]$. For $t\tau_5$, $DFC(t\tau_5)$ is calculated as 60 ms according to Equation (3). Since the timestamp $stamp[data(t\tau_5, 1)]$ is inherited from the execution start time of $\langle\tau_2, 1\rangle$, it is 0 ms. Thus, $j\tau_8$ jobs starting at or before $stamp[data(t\tau_5, 1)] + DFC(t\tau_5) = 60$ ms can utilize $data(t\tau_5, 1)$. However, $j\tau_8$ jobs beginning after this point cannot use the data. For example, $\langle j\tau_8, 2\rangle$, which starts at 100 ms, cannot access $data(t\tau_5, 1)$ or $data(t\tau_5, 2)$ but can use $data(t\tau_5, 3)$, as $stamp[data(t\tau_5, 3)] + DFC(t\tau_5) = 120$ ms.

In addition to *data freshness*, the system is also subject to an end-to-end deadline constraint, which limits the total time from input to output. Let D represent the end-to-end deadline for the exit node, and d_k denote the deadline for the kth job of the exit node. For instance, in Fig. 3, an end-to-end deadline of 100 ms is assigned to τ_9^{ed}. The deadline of the first job d_1 equals D, while deadlines for subsequent jobs are offset by their activation intervals. These activation intervals follow the period $T_{sg_{exit}}$ of the subgraph containing the exit node and are

Just Accepted

calculated as:

$$d_k = D + (k-1) \times T_{sg_{exit}}, \tag{4}$$

where $T_{sg_{exit}}$ is the period of the head *timer-driven node* in the subgraph.

3.3 Determining Job-level Dependencies

In an autonomous driving system, all jobs of the exit node must complete within the end-to-end deadline while ensuring that the *data freshness* constraints for each job are satisfied. To achieve this, it is crucial to analyze the relationships between jobs, referred to as job-level dependencies. These dependencies describe valid data exchanges, where data output by one job that meets the *data freshness* constraint is used by another job.

To capture all job-level dependencies, including those that cross hyper-period boundaries (i.e., data generated in one hyper-period and consumed in the next), the interval of analysis must be sufficiently long to reach a steady State. While the hyper-period (HP) represents the least common multiple (LCM) of the periods of subgraphs, analyzing a single HP is insufficient to capture dependencies where a job at the end of an HP passes data to a job at the beginning of the next HP. Therefore, to ensure that all successor relationships, and consequently the correct backward propagation of timing constraints, are identified, the analysis interval should be extended to two hyper-periods ($2 \times HP$). Extending the analysis interval to $2 \times HP$ is sufficient to capture all cross-hyper-period dependencies, because any data generated within one hyper-period can only be consumed at most in the immediately following hyper-period under the defined data freshness constraints. The hyper-period is calculated as follows:

$$HP = \mathrm{LCM}_{\forall \tau_i^{td} \in V^{td}}(T_i). \tag{5}$$

The first step in analyzing job-level dependencies is to compute reference start and finish times for each job. These reference times determine when data is read and written and help identify whether the data satisfies the *data freshness* constraint. By computing the reference start times (RST) and reference finish times (RFT) for all jobs within the extended interval $[0, 2 \times HP)$, it is possible to derive all job-level dependencies. The reference times for jobs of both *timer-driven nodes* and *event-driven nodes* are calculated using the equations provided in Ref. [17].

Once these reference times are established, the full set of job-level dependencies within the analysis interval can be determined. This enables tracking of data flow to each job of the exit node and identification of the specific jobs involved in this flow. Ultimately, the start times of these jobs can be used to estimate deadline miss probabilities at runtime.

The set of all job-level dependencies within the analysis interval is denoted as JLD, and a dependency from $\langle \tau_i, k \rangle$ to $\langle \tau_j, s \rangle$ is represented as $(\langle \tau_i, k \rangle \rightarrow \langle \tau_j, s \rangle) \in JLD$. Within each subgraph, job-level dependencies are straightforward to determine because node-level dependencies directly translate to job-level dependencies. For example, if a node τ_i has an edge to τ_j within the same subgraph, then $(\langle \tau_i, k \rangle \rightarrow \langle \tau_j, k \rangle)$ holds for any k. Consider the subgraph $sg_2 = \tau_2^{td}, \tau_5^{ed}$ in Fig. 3. The hyper-period HP of the DAG is 300 ms, as calculated from Equation (5). The number of times sg_2 executes during the analysis interval is given by:

$$N_{sg_i} = \frac{2 \times HP}{T_{sg_i}}. \tag{6}$$

Thus, $N_{sg_2} = \frac{2 \times 300}{30} = 20$, and the dependencies $(\langle \tau_2, 1 \rangle \rightarrow \langle \tau_5, 1 \rangle), (\langle \tau_2, 2 \rangle \rightarrow \langle \tau_5, 2 \rangle), \ldots, (\langle \tau_2, 20 \rangle \rightarrow \langle \tau_5, 20 \rangle)$ are added to JLD.

For dependencies between different subgraphs, i.e., from *tail nodes* to *join nodes*, the process is more complex. These dependencies are defined using reference times and *data freshness* constraints, as follows:

Definition 3.1 ([17]). Suppose that $e_{\tau_i,\tau_j} \in E$ and $t\tau_i$ is a tail node, and $j\tau_j$ is a join node. There exists a job-level dependency $(\langle t\tau_i, k \rangle \rightarrow \langle j\tau_j, s \rangle)$, iff

Just Accepted

Algorithm 1 Determination of Job-Level Dependencies

Input: Set of subgraphs SG, Hyper-period HP, Set of edges E
Output: Set of job-level dependencies JLD

$JLD \leftarrow \emptyset$
$RSTset \leftarrow \emptyset, RFTset \leftarrow \emptyset$
$I \leftarrow [0, 2 \times HP)$ ▷ Analysis interval
// Step 1: Instantiate jobs and compute reference times
for all $sg_i \in SG$ **do**
  $n \leftarrow N_{sg_i}$ ▷ Number of executions of sg_i in I (Equation (6))
  for $k \leftarrow 1 \ldots n$ **do**
    for all $\tau_a \in sg_i$ **do**
      Calculate $RST(\tau_a, k)$ and $RFT(\tau_a, k)$
      $RSTset \leftarrow RSTset \cup \{RST(\tau_a, k)\}$ ▷ Reference start time of each job
      $RFTset \leftarrow RFTset \cup \{RFT(\tau_a, k)\}$ ▷ Reference finish time of each job
    end for
    Add intra-subgraph dependencies to JLD
  end for
end for
// Step 2: Determine inter-subgraph dependencies
for all Pairs of *tail node* job $\langle t\tau_i, k\rangle$ and *join node* job $\langle j\tau_j, s\rangle$ with edges included in different subgraphs **do**
  if $RFT(t\tau_i, k) \in RFTset$ and $RST(j\tau_j, s) \in RSTset$ satisfy Conditions 1 and 2 in Theorem 3.1 **then**
    $JLD \leftarrow JLD \cup (\langle t\tau_i, k\rangle \rightarrow \langle j\tau_j, s\rangle)$
  end if
end for

(1) $RFT(t\tau_i, k) + comm_{i,j} \leq RST(j\tau_j, s)$ and
(2) $RST(j\tau_j, s) - stamp[data(t\tau_i, k)] \leq DFC(t\tau_i)$.

Here, let $\langle t\tau_i, k\rangle \in sg_i$, then $stamp[data(t\tau_i, k)]$ is given by:

$$stamp[data(t\tau_i, k)] = RST(\tau_{head(sg_i)}, k), \tag{7}$$

where $\tau_{head(sg_i)}$ is the head *timer-driven node* of the subgraph containing the job. Condition 1 ensures that data from the *tail node* arrives in time for the job of the *join node* reference start, while Condition 2 ensures the data satisfies the *data freshness* constraint.

The comprehensive procedure for constructing the job-level dependency graph is formalized in Algorithm 1. This algorithm extends the analysis interval to $2 \times HP$ to capture steady-state dependencies and identifies valid dependencies by verifying the timing and freshness conditions outlined in Theorem 3.1.

4 Proposed Method

Based on the job-level dependencies established in Section 3, the specific timing constraints for each job required to satisfy the end-to-end deadline of the system can now be determined. In conventional approaches to early detection, each job is assigned a specific time constraint, known as *laxity*, derived from these dependencies and the WCET of individual nodes. This *laxity* represents the available slack time relative to the end-to-end deadline; if the execution start time of a job exceeds its *laxity*, a deadline miss is predicted.

While the computation of *laxity* is straightforward when assuming fixed execution times (e.g., WCET) , such deterministic models are insufficient for real systems exhibiting execution time variability. Consequently, a

Just Accepted

Table 3. Notations for the proposed method

Symbol	Description
$L_{i,k}$	*plaxity* of $\langle \tau_i, k \rangle$
$\mathbb{L}_{i,k}$	Probability distribution of $L_{i,k}$
$L^{c}_{i,k}$	*plaxity-cdf* of $\langle \tau_i, k \rangle$
$\mathbb{L}^{c}_{i,k}$	Cumulative distribution of $L_{i,k}$
$AST(\tau_i, k)$	Actual start time of $\langle \tau_i, k \rangle$
$succ(\tau_i, k)$	Set of jobs with a job-level dependency from $\langle \tau_i, k \rangle$
$pred(\tau_i, k)$	Set of jobs with a job-level dependency to $\langle \tau_i, k \rangle$
th_{min}	Minimum probability threshold
th_{max}	Maximum probability threshold
$th_{i,k}$	Probability threshold of $\langle \tau_i, k \rangle$
$w_{i,k}$	Reference workload of $\langle \tau_i, k \rangle$
$rem(\tau_i, k)$	Remaining workload from $\langle \tau_i, k \rangle$ to the exit node job

static *laxity* model fails to account for the probabilistic nature of the workload, necessitating an extension to a probabilistic domain.

This section introduces probabilistic *laxity* (*plaxity*), an advanced methodology for calculating timing constraints in DAGs where nodes are characterized by probabilistic execution times. By accounting for inherent variability, this approach provides a more realistic estimation of job constraints. Furthermore, a methodology for determining the probability threshold necessary for deadline miss early detection using the calculated *plaxity* is provided. The notations relevant to this section are summarized in Table 3.

4.1 Probabilistic *laxity*

When execution times are treated as random variables, the computed *laxity* also becomes a random variable. This probabilistic extension of *laxity* is referred to as *plaxity*. The *plaxity* of a job $\langle \tau_i, k \rangle$ is denoted as $L_{i,k}$, and its probability distribution is represented by $\mathbb{L}_{i,k} = (\lambda_{i,k}, \ell p_{i,k})$. Here, $\lambda_{i,k,1}, \lambda_{i,k,2}, \ldots, \lambda_{i,k,u}$ specifies the possible values of *plaxity* in ascending order, while $\ell p_{i,k,1}, \ell p_{i,k,2}, \ldots, \ell p_{i,k,u}$ contains the corresponding probabilities. For instance, consider the following distribution:

$$\mathbb{L}_{i,k} = \begin{pmatrix} \lambda_{i,k} \\ \ell p_{i,k} \end{pmatrix} = \begin{pmatrix} 70 & 75 & 80 & 85 \\ 0.02 & 0.08 & 0.18 & 0.72 \end{pmatrix}.$$

In this example, the probability of $L_{i,k}$ being 70, 75, 80, and 85 is 0.02, 0.08, 0.18, and 0.72, respectively. The actual start time of a job, $AST(\tau_i, k)$, is compared with the probabilistic *laxity* to assess the likelihood of meeting or missing a deadline. For example, if $75 < AST(\tau_i, k) \leq 80$, the probability $P(L_{i,k} < AST(\tau_i, k))$ is calculated as $P(L_{i,k} = 70) + P(L_{i,k} = 75)$. Thus, $P(L_{i,k} < AST(\tau_i, k)) = 0.02 + 0.08 = 0.1$. The probability $P(AST(\tau_i, k) \leq L_{i,k})$ is $1 - P(L_{i,k} < AST(\tau_i, k)) = 0.9$. Therefore, the probability of missing the deadline is 0.1, while the probability of meeting the deadline is 0.9.

The threshold for the probability of meeting the deadline is determined according to user and system requirements. If the computed probability of meeting the deadline drops below this predefined threshold, it signals the detection of a deadline miss. However, as calculated above, the probability $P(AST(\tau_i, k) \leq L_{i,k})$ cannot be directly obtained by comparing $AST(\tau_i, k)$ and $L_{i,k}$ in its original form.

Just Accepted

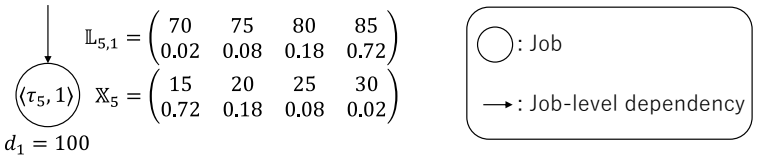


Fig. 5. *plaxity* of the exit node job.

To address this, *plaxity-cdf* is defined as the cumulative probability form of *plaxity*, denoted as $L^{c}_{i,k}$. The distribution of $L^{c}_{i,k}$ is represented as $\mathbb{L}^{c}_{i,k} = (\lambda_{i,k}, \ell p^{c}_{i,k})$. Here, for all integers v with $1 \leq v \leq u$, $\ell p^{c}_{i,k,v}$ is computed as follows:

$$\ell p^{c}_{i,k,v} = \sum_{h=v}^{u} \ell p_{i,k,h}. \tag{8}$$

For the earlier example, the cumulative form is computed as:

$$\mathbb{L}^{c}_{i,k} = \begin{pmatrix} \lambda_{i,k} \\ \ell p^{c}_{i,k} \end{pmatrix} = \begin{pmatrix} 70 & 75 & 80 & 85 \\ 1 & 0.98 & 0.9 & 0.72 \end{pmatrix}.$$

By comparing $AST(\tau_i, k)$ and $\mathbb{L}^{c}_{i,k}$, the probability of meeting the deadline can be directly determined based on the start time of the job. Specifically, for $AST(\tau_i, k)$ such that $\lambda_{i,k,v-1} < AST(\tau_i, k) \leq \lambda_{i,k,v}$, the corresponding probability of meeting the deadline is $\ell p^{c}_{i,k,v}$. It is important to note the following: $\lambda_{i,k,0} = 0$, $\lambda_{i,k,u+1} = \infty$, and $\ell p^{c}_{i,k,u+1} = 0$. The *plaxity-cdf* serves primarily as a comparative measure, enabling a straightforward assessment of the probability of meeting deadlines directly from job start times. However, it is not directly involved in the recursive computation of plaxity values for individual jobs. Instead, the computation of *plaxity* relies on the probabilistic execution times of jobs and their dependencies within the task graph, ensuring precise characterization of timing constraints.

In the following, we describe the method for calculating the *plaxity* for each job. Similar to *laxity*, the *plaxity* is computed recursively, beginning with the job of the exit node. Hence, the method for deriving the *plaxity* for the job at the exit node is outlined first. The potential values of *plaxity* for this job are obtained by subtracting possible execution times from the specified deadline. The probability of each *plaxity* value matches the probability of the job having the corresponding execution time. The *plaxity* $L_{i,k}$ for the job at the exit node, denoted as $\langle \tau_i, k \rangle$, is rigorously defined as follows:

$$P(L_{i,k} = \lambda) = P(X_i = d_k - \lambda). \tag{9}$$

An example of calculating the *plaxity* $L_{5,1}$ for the job of the exit node $\langle \tau_5, 1 \rangle$ in Fig. 5 is provided below. Using $\mathbb{X}_5$, d_k, and Equation (9), the following probabilities are derived:

$$\begin{aligned} P(L_{5,1} = 85) &= P(X_5 = 15) = 0.72, \\ P(L_{5,1} = 80) &= P(X_5 = 20) = 0.18, \\ P(L_{5,1} = 75) &= P(X_5 = 25) = 0.08, \\ \text{and} \quad P(L_{5,1} = 70) &= P(X_5 = 30) = 0.02. \end{aligned}$$

Therefore, $\mathbb{L}_{5,1}$ is represented as:

$$\mathbb{L}_{5,1} = \begin{pmatrix} 70 & 75 & 80 & 85 \\ 0.02 & 0.08 & 0.18 & 0.72 \end{pmatrix}.$$

Next, the *plaxity* of jobs with successor jobs is defined. For clarity, this explanation begins with jobs that have only one successor. The *plaxity* of such jobs is determined by applying the convolution operation introduced in

Just Accepted

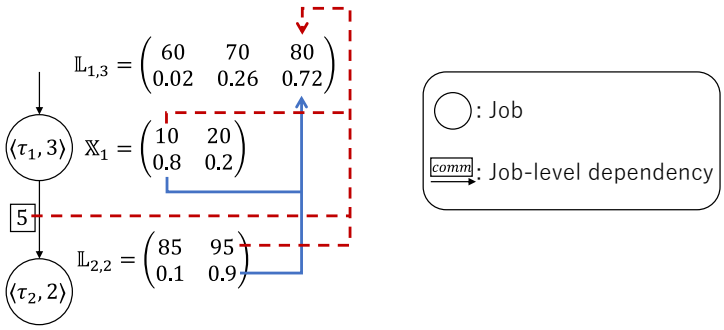


Fig. 6. *plaxity* of the job that has one successor job.

Equation (1). The possible values of the *plaxity* are calculated by subtracting the communication time between the jobs and the execution time of the predecessor job from the *plaxity* values of the successor job. The probability of each *plaxity* value corresponds to the joint probability of the predecessor job having each execution time and the successor job taking the corresponding *plaxity* value. As the random variables are assumed to be independent, their joint probability is the product of the individual probabilities. We now introduce the concept of *convolutional subtraction* as applied to Equation (1). This operation calculates the *plaxity* of a job $\langle \tau_i, k \rangle$ with a single successor job $\langle \tau_j, s \rangle$. The *plaxity* is derived using convolutional subtraction, expressed as $L_{i,k} = L_{j,s} \otimes^{-} X_i$ and formally defined as:

$$P(L_{i,k} = \lambda) = \sum_{m=1}^{l} P(X_i = x_{i,m}) \cdot P(L_{j,s} = \lambda + comm_{i,j} + x_{i,m}). \tag{10}$$

Note that this operation considers the communication time between jobs. An example of calculating the *plaxity* $L_{1,3}$ for the job $\langle \tau_1, 3 \rangle$ is shown in Fig. 6. Using $\mathbb{X}_1$, $comm_{1,2}$, $\mathbb{L}_{2,2}$, and Equation (10), the following probabilities are computed:

$$\begin{aligned} P(L_{1,3} = 60) &= P(X_1 = 20) \cdot P(L_{2,2} = 85) = 0.02, \\ P(L_{1,3} = 70) &= P(X_1 = 10) \cdot P(L_{2,2} = 85) \\ &\quad + P(X_1 = 20) \cdot P(L_{2,2} = 95) = 0.26, \\ \text{and} \quad P(L_{1,3} = 80) &= P(X_1 = 10) \cdot P(L_{2,2} = 95) = 0.72. \end{aligned}$$

Therefore, $\mathbb{L}_{1,3}$ is expressed as:

$$\mathbb{L}_{1,3} = \begin{pmatrix} 60 & 70 & 80 \\ 0.02 & 0.26 & 0.72 \end{pmatrix}.$$

Finally, the *plaxity* for jobs with multiple successors is defined. In the case of *laxity*, to satisfy the deadlines of all subsequent paths, the minimum *laxity* among successor jobs is used. However, for *plaxity*, a direct comparison is not feasible, and determining the minimum requires additional considerations.

Suppose there are two successor jobs, each associated with a different *plaxity*. A new *plaxity* is defined as the probability that both jobs meet their deadlines when starting at the same time t. This probability represents the joint probability of each job independently satisfying its deadline from t. Since the *plaxity* values of the jobs are assumed to be independent, their joint probability is calculated as the product of the respective probabilities. Therefore, the minimum value L of two *plaxity* values, $L_{i,k}$ and $L_{j,s}$, is expressed as:

$$\begin{aligned} \min(L_{i,k}, L_{j,s}) &= L; \\ P(L = \lambda) = P(L_{i,k} = \lambda) \cdot P(\lambda \leq L_{j,s}) &+ P(\lambda < L_{i,k}) \cdot P(L_{j,s} = \lambda). \end{aligned} \tag{11}$$

Just Accepted

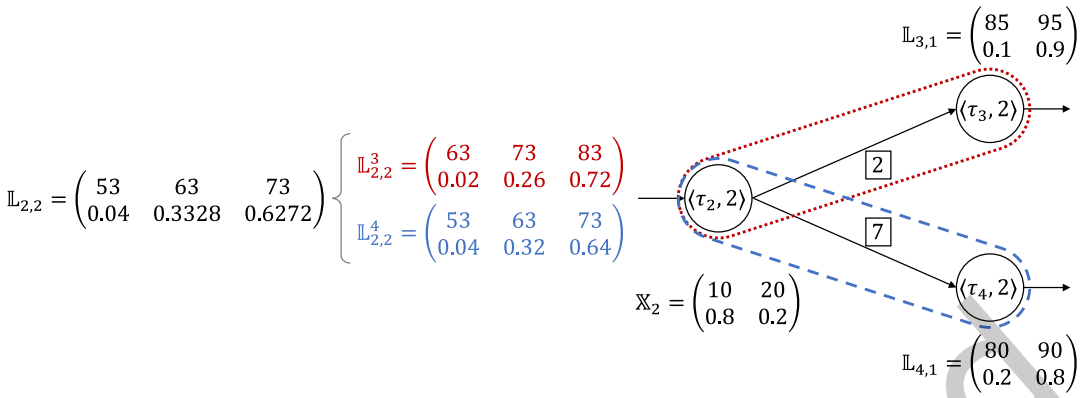


Fig. 7. *plaxity* of the job that has multiple successor jobs.

The minimum value of *plaxity* for three or more jobs can be computed by iteratively applying this calculation. Thus, the *plaxity* $L_{i,k}$ for the job $\langle \tau_i, k \rangle$ with multiple successor jobs is determined using Equations (10) and (11), as follows:

$$L_{i,k} = \min_{\langle \tau_j, s \rangle \in succ(\tau_i, k)} \left(L_{j,s} \otimes^{-} X_i \right). \tag{12}$$

An illustrative example of calculating the *plaxity* $L_{2,2}$ for a job $\langle \tau_2, 2 \rangle$ with two successor jobs is shown in Fig. 7. First, the intermediate *plaxity* derived from each successor path, denoted as $\mathbb{L}^3_{2,2}$ (red) and $\mathbb{L}^4_{2,2}$ (blue), are calculated separately using Equation (10), following the same procedure illustrated in Fig. 6.

Next, the combined *plaxity* $\mathbb{L}_{2,2}$ is determined by taking the minimum of these two distributions using Equation (11). Using $\mathbb{L}^3_{2,2}$, $\mathbb{L}^4_{2,2}$, and Equation (11), the following probabilities are computed:

$$\begin{aligned} P(L_{2,2} = 53) &= P(L^3_{2,2} = 53) \cdot P(L^4_{2,2} \geq 53) + P(L^4_{2,2} = 53) \cdot P(L^3_{2,2} > 53) = 0.04, \\ P(L_{2,2} = 63) &= P(L^3_{2,2} = 63) \cdot P(L^4_{2,2} \geq 63) + P(L^4_{2,2} = 63) \cdot P(L^3_{2,2} > 63) = 0.3328, \\ P(L_{2,2} = 73) &= P(L^3_{2,2} = 73) \cdot P(L^4_{2,2} \geq 73) + P(L^4_{2,2} = 73) \cdot P(L^3_{2,2} > 73) = 0.6272, \\ \text{and} \quad P(L_{2,2} = 83) &= P(L^3_{2,2} = 83) \cdot P(L^4_{2,2} \geq 83) + P(L^4_{2,2} = 83) \cdot P(L^3_{2,2} > 83) = 0. \end{aligned}$$

Therefore, $\mathbb{L}_{2,2}$ is represented as:

$$\mathbb{L}_{2,2} = \begin{pmatrix} 53 & 63 & 73 \\ 0.04 & 0.3328 & 0.6272 \end{pmatrix}.$$

The *plaxity* for all jobs is computed using Equations (9) and (12). Applying Equation (8) to each computed *plaxity* yields the corresponding *plaxity-cdf*.

4.2 Probability Threshold

During runtime, the actual start time of the job and the *plaxity-cdf* are compared, and if the probability of meeting the deadline is below the threshold, a deadline miss is detected. In practice, probability thresholds can be set to arbitrary values for each job. However, this paper proposes a method for determining probability thresholds that enables effective early detection of a potential deadline miss. The proposed method is fundamentally based on the concept of gradually tightening the thresholds. The rationale for this strategy is clear. Consider a scenario where the probability of meeting the deadline is low at the beginning of a job. If subsequent jobs are completed in a short amount of time, the probability of meeting the deadline can potentially increase compared to the initial

Just Accepted

assessment. Such occurrences are more likely as the distance from the exit node job increases, meaning that there are more subsequent jobs. In this case, setting the same threshold for all jobs would lead to pessimistic detection. Therefore, the minimum and maximum thresholds are set arbitrarily, with the probability threshold of the entry node job set to the minimum value and higher probability thresholds applied as the exit node job is approached. Specifically, in this approach, the probability threshold increases linearly with respect to the remaining workload until the exit node job.

This paper evaluates two approaches for defining the workload for determining probability thresholds. In the first approach, the workload is uniformly defined such that every job contributes equally. Formally, the workload of each job $\langle \tau_i, k \rangle$ is defined as follows:

$$w_{i,k} = 1. \tag{13}$$

In this case, the remaining workload at a given job simply corresponds to the number of jobs until the exit node job.

In the second approach, the workload is defined based on the average execution time of jobs. Formally, the workload of each job $\langle \tau_i, k \rangle$ is defined as follows:

$$w_{i,k} = E(X_i), \tag{14}$$

where $E(X_i)$ represents the mean of the random variable X_i, given by $E(X_i) = \sum_{j=1}^{l} x_{i,j} \cdot p_{i,j}$. In this case, the remaining workload for a given job corresponds to the mean of the total execution times of all jobs up to the exit node job. These two methods allow for a nuanced adjustment of the probability thresholds and are compared in the evaluation to assess their impact on detection accuracy.

The probability threshold for each job is defined as follows. First, the probability threshold $th_{i,k}$ for the entry node job $\langle \tau_i, k \rangle$ is set to the minimum value:

$$th_{i,k} = th_{min}, \tag{15}$$

where th_{min} is the minimum value of the probability threshold and can be arbitrarily assigned. Similarly, the probability threshold $th_{i,k}$ for the exit node job $\langle \tau_i, k \rangle$ is set to the maximum value:

$$th_{i,k} = th_{max}, \tag{16}$$

where th_{max} is the maximum value of the probability threshold and can also be arbitrarily assigned, but $th_{min} < th_{max}$ must hold.

Next, the probability thresholds for other jobs are defined. As described earlier, the thresholds vary according to the remaining workload until the exit node job, which must be determined. The remaining workload for a given job $\langle \tau_i, k \rangle$ is obtained as follows:

$$rem(\tau_i, k) = \min_{\langle \tau_j, s \rangle \in succ(\tau_i, k)} rem(\tau_j, s) + w_{i,k}. \tag{17}$$

By calculating iteratively from the exit node job, the remaining workload can be determined for all jobs. Note that the remaining workload for the exit node job is the workload of itself. The minimum is taken to prioritize paths with smaller remaining workload, as this results in a higher probability threshold.

Using the above information, the probability threshold $th_{i,k}$ for a job $\langle \tau_i, k \rangle$ is defined as:

$$th_{i,k} = \max_{\langle \tau_j, s \rangle \in pred(\tau_i, k)} th_{j,s} + \frac{th_{max} - th_{j,s}}{rem(\tau_i, k)}. \tag{18}$$

The calculation proceeds iteratively from the entry node job, defining the probability thresholds for all jobs. The maximum is taken to prioritize the path with the highest probability threshold when multiple preceding jobs exist.

Just Accepted

The rationale for adopting a linear relationship between the probability threshold and the remaining workload is based on the heuristic assumption that execution time uncertainty resolves uniformly as the process progresses toward the deadline. In the absence of specific prior knowledge regarding the distribution of variance across the task chain (e.g., whether early tasks are more volatile than later ones), a linear interpolation serves as the most neutral and robust strategy. An important property of this linear interpolation is monotonicity: the probability threshold is guaranteed to be non-decreasing as the remaining workload decreases. This ensures that the detection condition becomes strictly more conservative as the system approaches the exit node, preventing counter-intuitive threshold relaxation near the deadline. Adopting a non-linear function, such as an exponential or logarithmic function, would imply a specific bias regarding where variability is concentrated, which could limit the generality of the method.

Crucially, while the interpolation function itself is linear, the definition of "workload" incorporates a mechanism for weighting. By defining the workload based on average execution times (Equation (14)) rather than a simple job count (Equation (13)), the threshold adjustment implicitly assigns greater weight to tasks with longer durations, which typically exhibit larger absolute variance. This ensures that the detection sensitivity is appropriately tuned to the magnitude of the remaining uncertainty, as demonstrated by the superior performance of the time-based approach in Section 5.

The effect of relaxing probability thresholds for early-stage nodes must be carefully considered to ensure the preservation of worst-case safety guarantees. In our system model, the ultimate safety guarantee is provided by the MRM, which transitions the vehicle to a fail-safe state upon a confirmed deadline miss. The role of early detection is to trigger this MRM sooner to enhance safety margins, not to replace the final deadline check.

Using a fixed, conservative threshold (e.g., based on WCET) at early stages often leads to excessive false positives due to the high variance of remaining execution times. This would cause frequent, unnecessary MRM transitions, rendering the autonomous driving system practically unusable. By relaxing the threshold when the remaining workload is large, and progressively tightening it as the deadline approaches, the proposed method balances system availability with detection sensitivity. This design rationale is empirically validated in Section 5, where the experimental results demonstrate that our variable threshold approach reduces false positives (improving Precision) while maintaining a high detection rate (Recall) comparable to conservative WCET-based methods.

5 Evaluation

The performance of the proposed method for early detection of deadline miss is evaluated through experiments which are performed by simulating task scheduling for multiple periodic DAGs.

5.1 Experiment Setup

The evaluation experiments focus on DAGs generated randomly by RD-Gen [18], with adjustments applied to align with the task model analyzed in this study. The periods of *timer-driven nodes* and communication times are initialized with default settings. Initially, two types of edges, *trigger edges* and *update edges*, are defined. Since *timer-driven nodes* operate based on their predefined periods, all input edges for these nodes are classified as *update edges*, and there are no *trigger edges* connected to them. For *event-driven nodes*, if a node has exactly one input edge, that edge is designated as a *trigger edge*. If multiple input edges exist, the edge type is determined according to the *chain* information provided by RD-Gen. Edges originating from nodes within the same chain are labeled as *trigger edges*, whereas inputs from nodes outside the chain are categorized as *update edges*. It is important to note that *event-driven nodes* may possess multiple *trigger edges*. Such nodes remain idle until all preceding nodes connected via *trigger edges* have completed execution.

The execution times for individual nodes are assigned using the specified WCET values. The average execution time is defined as $\mu = WCET/3.0$, and the standard deviation is set to $\sigma = \mu/2.0$. Execution times predominantly

Just Accepted

follow a normal distribution $\mathcal{N}(\mu, \sigma)$ with a probability of 0.98, while with a probability of 0.02, they adhere to a normal distribution $\mathcal{N}(WCET, 0.2\sigma)$. This configuration is derived from the statistical analysis of real-world *Autoware* execution traces. The data was collected during autonomous driving tests on actual vehicles and profiled using CARET (Chain-Aware ROS Evaluation Tool) [8]. CARET enables precise measurement of execution times at the node granularity with minimal overhead. Our analysis of the empirical data revealed a bi-modal distribution pattern: a dominant mode centered around the average execution time (approx. 1/3 of the observed worst-case) representing nominal operation, and a tail mode approaching the worst-case latency due to system overheads and resource contention. To faithfully reproduce this behavior in our simulation, we adopt the mixture of two normal distributions described above.

The scheduling algorithm used in this study is Earliest Deadline First (EDF) [13]. In this approach, jobs closer to their deadlines are given higher priority for execution. Implicit deadlines are assumed, where the relative deadline of each job matches the period of its corresponding node. For *event-driven nodes* lacking their own periods, the relative deadline is set equal to the period of the subgraph they belong to. Furthermore, all jobs are executed in a non-preemptive manner. Once a processor is allocated to a job, the execution continues uninterrupted until the job is complete.

The simulations are implemented using Python and executed on a machine equipped with a 2.9 GHz 8-Core Intel Core i7 CPU, 32 GiB RAM, and running Ubuntu 22.04 LTS (64-bit) OS.

5.2 Performance Metrics

This section introduces the metrics used to evaluate the performance of the proposed method. The simulation results are classified into the following categories to compute these metrics:

- *True Positive (TP):* TP occurs when a deadline miss is detected early by the proposed method, and an actual deadline miss happens during further processing. This result validates the accuracy of the early detection mechanism.
- *False Positive (FP):* FP arises when the proposed method detects a deadline miss early, but no actual deadline miss occurs. This indicates a pessimistic detection. In autonomous driving systems, frequent FP cases may cause unnecessary transitions to the Minimum Risk Maneuver (MRM) mode, disrupting normal operations.
- *True Negative (TN):* TN is when no deadline miss is detected early, and no deadline miss occurs during processing. This result signifies correct prediction when the system is in a safe state. In safety-critical real-time systems, deadline misses are rare, making TN a prevalent outcome when early detection is highly accurate.
- *False Negative (FN):* FN occurs when no deadline miss is detected early, but an actual deadline miss happens during processing. This reflects the inability of the proposed method to detect the deadline miss in advance.

Using these classifications, the following performance metrics are derived to assess the effectiveness of the proposed method:

- ***Accuracy:*** *Accuracy* quantifies the proportion of correct predictions among all predictions and is calculated as:

$$Accuracy = \frac{TP + TN}{TP + FP + TN + FN} \tag{19}$$

  A value closer to 1 indicates fewer FP and FN cases, signifying higher accuracy.
- ***Recall:*** *Recall* measures the fraction of actual deadline misses that are predicted and is defined as:

$$Recall = \frac{TP}{TP + FN} \tag{20}$$

  High *Recall* values are desirable from a safety perspective, ensuring most deadline misses are reliably detected.

Just Accepted

Table 4. Experimental parameters in Section 5.3

Parameter	Value
Number of cores	8
Normalized utilization [%]	275, 280, 285, 290, 295, 300, 305[1]
α in Equation (3)	2.0, 2.1, 2.2, 2.3, 2.4, 2.5
Number of entry nodes	7, 8, 9
Number of exit nodes	1
Period of *timer-driven nodes* [ms]	10, 20, 30, 50, 60, 100
Probability threshold	0.7-1.0, 0.8-1.0, 0.9-1.0, 0.95, 0.99, 1.0

- ***Precision:*** *Precision* evaluates the proportion of accurate deadline miss predictions among all predictions and is calculated as:

$$Precision = \frac{TP}{TP + FP} \tag{21}$$

  A higher *Precision* indicates fewer unnecessary disruptions due to false alarms.
- ***F-measure:*** *F-measure* is the harmonic mean of *Recall* and *Precision*, balancing these two metrics:

$$F\text{-}measure = 2 \cdot \frac{Recall \cdot Precision}{Recall + Precision} \tag{22}$$

  This metric helps evaluate the trade-off between *Recall* and *Precision*, aiming for an optimal balance.

5.3 Comparison of Detection Results with Different Thresholds

In this section, the performance of deadline miss early detection using dynamically adjusted probability thresholds is examined. This method incorporates workload-based adjustments to improve prediction accuracy.

The early detection of deadline miss is conducted at the beginning of job execution by referencing the *plaxity-cdf*. Specifically, the largest *plaxity-cdf* value with a probability greater than or equal to the dynamically adjusted threshold is compared with the execution start time. If the execution start time exceeds this value, a deadline miss is detected. Conversely, if the probability of meeting the deadline at the execution start time is above the adjusted threshold, no deadline miss is detected.

The dynamic threshold is determined based on the remaining workload $rem(\tau_i, k)$ until the exit job. Two workload calculation methods are evaluated: one where each job contributes equally, and another where the contribution is proportional to the average execution time of jobs. The thresholds increase progressively as the workload decreases, resulting in stricter predictions for jobs closer to the exit node. Adjustment of thresholds provides greater flexibility in balancing detection sensitivity and false alarms.

Note that the configuration with a probability threshold of 1.0 exactly reproduces the deterministic WCET-based early-detection method [17]. Therefore, the comparison between the proposed method and the threshold of 1.0 effectively serves as a direct comparison with the existing state-of-the-art method for mixed timer-driven and event-driven DAGs.

For each probability threshold, simulations are performed with increasing task utilization, and the results are evaluated using defined performance metrics. The parameters used in this section are listed in Table 4.

[1]For a meaningful comparison of deadline miss prediction performance, it is necessary to analyze scenarios where deadline misses actually occur. In this experimental setup, normalized utilization is calculated based on WCET, even though actual job execution times are significantly shorter than WCET. Consequently, task sets with normalized utilization far exceeding 100% are examined to ensure that deadline misses are observed.

Just Accepted

The normalized utilization is the total utilization of the task divided by the number of processor cores. The task utilization is based on the assumption of WCET. Note that normalized utilization above 100% does not immediately imply system overload, as actual execution times are significantly smaller than WCET in this study. In this experiment, utilization is increased to a level where deadline misses occur, facilitating comparison, and this often exceeds 100%.

The variations in TP, FP, TN, and FN ratios with increasing utilization are depicted in Fig. 8. Here, the proposed methods include (job), which determines thresholds based on the number of jobs, and (time), which determines thresholds based on the average execution time. For every utilization level, corresponding to each point on the x-axis of the figures, 500 randomly generated DAGs were executed 10 times. It is important to highlight that the execution times of nodes vary across runs even for the same DAG. With higher utilization rates, the frequency of deadline misses grows. When comparing the proposed method with cases where the threshold is set to 1.0 (i.e., using a fixed WCET) or a global threshold of 0.99, it is evident that the proposed method can reduce FP and mitigate pessimism in detection. Although the difference between the proposed method and a global threshold of 0.95 initially seems minor, the proposed method demonstrates its effectiveness in reducing FP in scenarios with low utilization rates, where the deadline miss rate is low. The overall change in proportion might appear small, but in situations where deadline misses are infrequent, even a slight reduction in FP can have a significant impact. The actual influence of FP reduction in low deadline miss rate cases can be evaluated using the previously defined Precision metric, which will be discussed later.

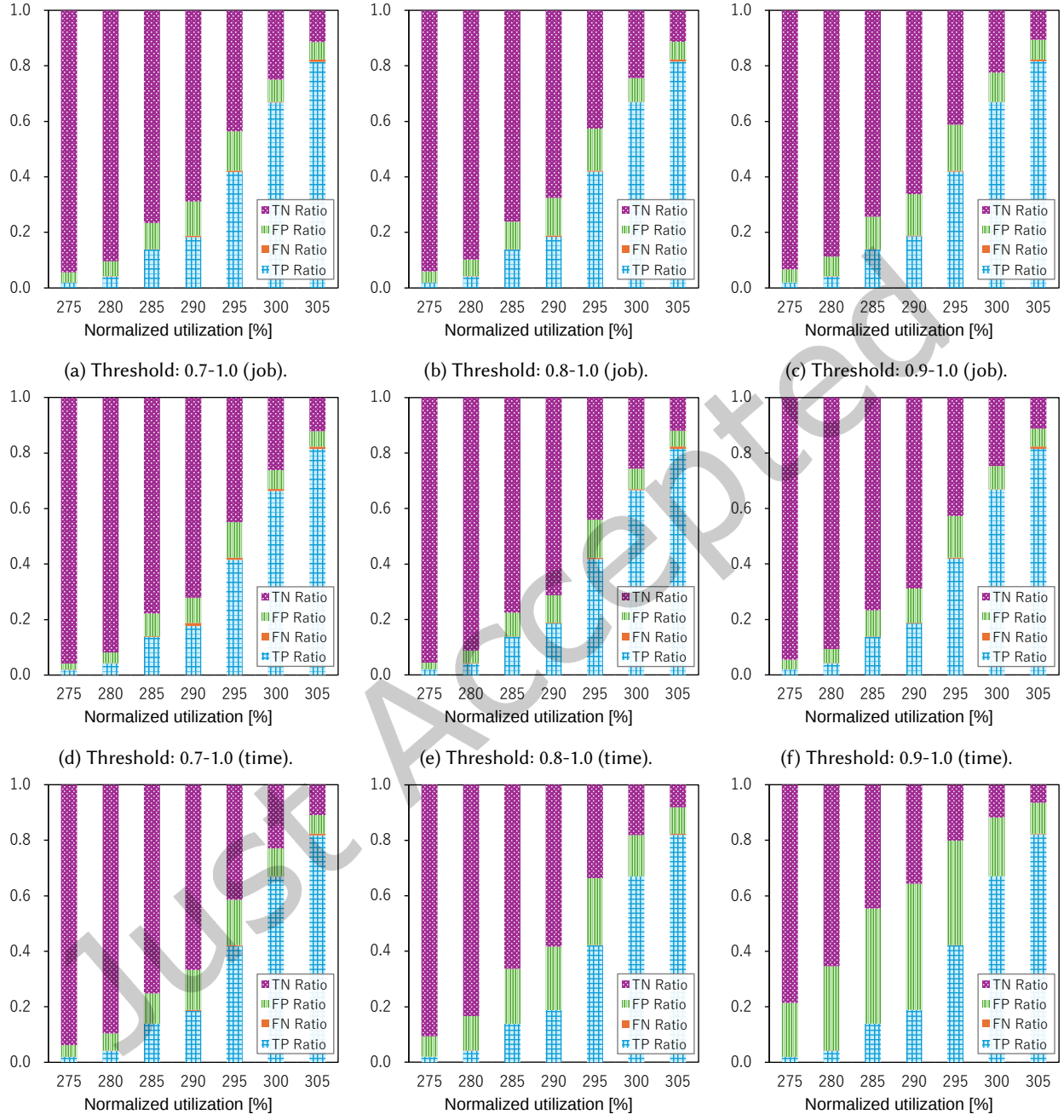
Lowering the threshold minimum increases the FN count. In this evaluation, since the maximum threshold is set to 1, an increase in FN compared to cases with a global threshold below 1 seems counterintuitive. This phenomenon arises when the threshold fails to sufficiently increase near the end of the process chain. Such situations occur when paths include an extremely small number of jobs from the entry to the exit job, or when the average execution time of nodes near the exit is exceptionally large. Adopting a nonlinear increase in probabilistic thresholds is expected to enable appropriately higher thresholds for these cases, leading to further improvements in accuracy.

Then, the *Accuracy*, *Precision*, *Recall*, and *F-measure* are calculated from these values of TP, FP, TN, and FN. The evaluation results of these metrics are shown in Fig. 9. *Accuracy* improves significantly when compared to thresholds set at 1.0 or 0.99, but shows almost identical results when compared to a threshold set at 0.95. Further detailed analysis is performed using additional metrics. *Recall*, which represents the proportion of actual deadline misses detected, remains close to 1 across all methods, minimizing undetected misses. However, as mentioned earlier, lowering the minimum threshold noticeably reduces this value. Therefore, care must be taken when setting the lower limit of the threshold to ensure system safety. When the threshold lower limit is set to 0.8, the method does not fall below the results of the method with a global threshold of 0.95. *Precision*, which indicates the proportion of detected deadline misses that were actual misses, varies significantly across methods. When thresholds are set globally to 1 or 0.99, particularly in cases of low deadline miss rates, *Precision* becomes extremely low. Comparing the method with a global threshold of 0.95 and the proposed method using a lower limit of 0.8 with thresholds determined based on average execution time, the proposed method achieves up to 1.5 times higher *Precision*. As mentioned earlier, since *Recall* for these methods remains almost identical, pessimism is reduced without compromising reliability. Among the two approaches for determining thresholds in the proposed method, the approach based on average execution time achieves higher values.

F-measure, the harmonic mean of *Precision* and *Recall*, shows trends similar to *Precision*, as *Recall* is nearly 1 across all methods. Consequently, the method that determines thresholds based on average execution time with a lower limit of 0.8 again demonstrates superior values.

To validate the statistical robustness of the observed improvements, the distribution of performance metrics across the 500 generated DAGs was analyzed. The 95% confidence intervals for *Accuracy*, *Recall*, *Precision*, and

Just Accepted



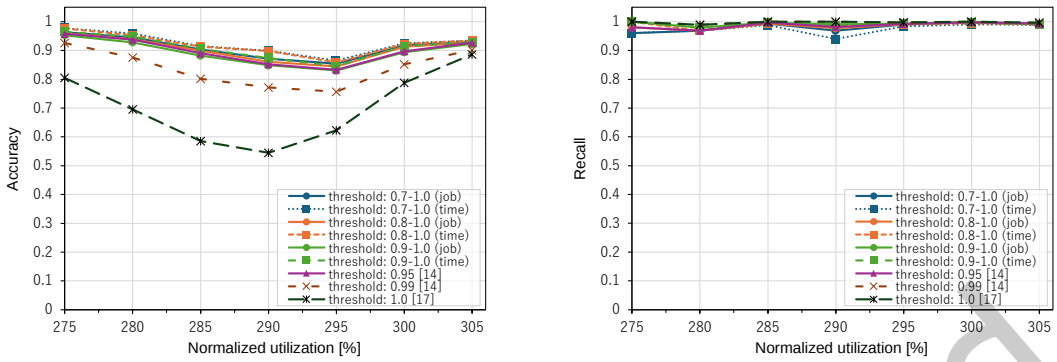
(a) Threshold: 0.7-1.0 (job). (b) Threshold: 0.8-1.0 (job). (c) Threshold: 0.9-1.0 (job).

(d) Threshold: 0.7-1.0 (time). (e) Threshold: 0.8-1.0 (time). (f) Threshold: 0.9-1.0 (time).

(g) Threshold: 0.95. (h) Threshold: 0.99. (i) Threshold: 1.0.

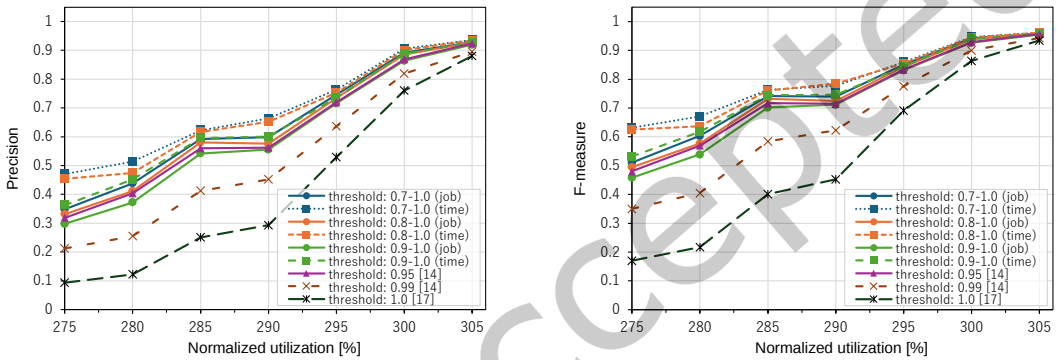
Fig. 8. TP, FP, TN, and FN ratios with increasing utilization.

Just Accepted



(a) *Accuracy*.

(b) *Recall*.



(c) *Precision*.

(d) *F-measure*.

Fig. 9. *Accuracy*, *Recall*, *Precision* and *F-measure* results with increasing utilization.

Table 5. 95% confidence interval of *Accuracy*

	Normalized utilization						
Threshold	275%	280%	285%	290%	295%	300%	305%
0.7-1.0 (job)	[0.960, 0.965]	[0.943, 0.949]	[0.903, 0.909]	[0.869, 0.874]	[0.852, 0.858]	[0.915, 0.920]	[0.927, 0.933]
0.7-1.0 (time)	[0.976, 0.981]	[0.957, 0.963]	[0.911, 0.916]	[0.895, 0.900]	[0.864, 0.869]	[0.921, 0.927]	[0.933, 0.939]
0.8-1.0 (job)	[0.958, 0.964]	[0.939, 0.945]	[0.896, 0.902]	[0.859, 0.864]	[0.842, 0.847]	[0.910, 0.915]	[0.922, 0.928]
0.8-1.0 (time)	[0.972, 0.977]	[0.951, 0.956]	[0.907, 0.913]	[0.895, 0.901]	[0.856, 0.862]	[0.919, 0.925]	[0.933, 0.939]
0.9-1.0 (job)	[0.949, 0.954]	[0.927, 0.932]	[0.878, 0.884]	[0.847, 0.852]	[0.829, 0.835]	[0.890, 0.896]	[0.916, 0.921]
0.9-1.0 (time)	[0.962, 0.968]	[0.945, 0.950]	[0.902, 0.908]	[0.870, 0.875]	[0.845, 0.851]	[0.912, 0.918]	[0.926, 0.932]
0.95 [14]	[0.959, 0.964]	[0.935, 0.940]	[0.889, 0.895]	[0.850, 0.856]	[0.832, 0.838]	[0.895, 0.901]	[0.927, 0.933]
0.99 [14]	[0.920, 0.925]	[0.874, 0.879]	[0.799, 0.805]	[0.771, 0.776]	[0.755, 0.761]	[0.849, 0.854]	[0.898, 0.903]
1.0 [17]	[0.802, 0.808]	[0.693, 0.698]	[0.583, 0.588]	[0.545, 0.551]	[0.618, 0.624]	[0.786, 0.791]	[0.884, 0.890]

F-measure were calculated and are presented in Tables 5 to 8, respectively. As indicated in tables, the intervals are extremely narrow, indicating highly stable performance across diverse DAG structures.

Just Accepted

Table 6. 95% confidence interval of *Recall*

	Normalized utilization						
Threshold	275%	280%	285%	290%	295%	300%	305%
0.7-1.0 (job)	[0.960, 0.962]	[0.967, 0.970]	[0.994, 0.997]	[0.969, 0.971]	[0.989, 0.992]	[0.995, 0.998]	[0.988, 0.991]
0.7-1.0 (time)	[0.958, 0.961]	[0.968, 0.970]	[0.987, 0.990]	[0.937, 0.940]	[0.982, 0.985]	[0.987, 0.990]	[0.989, 0.992]
0.8-1.0 (job)	[0.979, 0.981]	[0.965, 0.968]	[0.992, 0.994]	[0.975, 0.978]	[0.989, 0.991]	[0.997, 1.000]	[0.989, 0.992]
0.8-1.0 (time)	[0.999, 1.000]	[0.966, 0.969]	[0.992, 0.995]	[0.983, 0.986]	[0.988, 0.991]	[0.993, 0.996]	[0.989, 0.992]
0.9-1.0 (job)	[1.000, 1.000]	[0.978, 0.981]	[0.996, 0.999]	[0.990, 0.993]	[0.991, 0.994]	[0.998, 1.000]	[0.992, 0.995]
0.9-1.0 (time)	[0.999, 1.000]	[0.978, 0.981]	[0.996, 0.998]	[0.985, 0.987]	[0.991, 0.994]	[0.994, 0.997]	[0.988, 0.991]
0.95 [14]	[0.978, 0.981]	[0.966, 0.969]	[0.996, 0.999]	[0.980, 0.983]	[0.990, 0.993]	[0.997, 1.000]	[0.990, 0.993]
0.99 [14]	[0.999, 1.000]	[0.988, 0.990]	[0.999, 1.000]	[0.999, 1.000]	[0.994, 0.997]	[0.999, 1.000]	[0.994, 0.996]
1.0 [17]	[0.999, 1.000]	[0.988, 0.991]	[1.000, 1.000]	[0.999, 1.000]	[0.997, 0.999]	[0.999, 1.000]	[0.996, 0.999]

Table 7. 95% confidence interval of *Precision*

	Normalized utilization						
Threshold	275%	280%	285%	290%	295%	300%	305%
0.7-1.0 (job)	[0.346, 0.351]	[0.434, 0.440]	[0.592, 0.597]	[0.598, 0.603]	[0.745, 0.750]	[0.890, 0.895]	[0.927, 0.933]
0.7-1.0 (time)	[0.468, 0.474]	[0.514, 0.520]	[0.618, 0.624]	[0.663, 0.668]	[0.759, 0.765]	[0.902, 0.907]	[0.935, 0.940]
0.8-1.0 (job)	[0.329, 0.334]	[0.406, 0.412]	[0.577, 0.583]	[0.572, 0.578]	[0.731, 0.736]	[0.885, 0.890]	[0.923, 0.929]
0.8-1.0 (time)	[0.449, 0.455]	[0.469, 0.474]	[0.613, 0.618]	[0.650, 0.656]	[0.751, 0.756]	[0.896, 0.901]	[0.931, 0.936]
0.9-1.0 (job)	[0.296, 0.301]	[0.369, 0.375]	[0.540, 0.545]	[0.551, 0.557]	[0.713, 0.719]	[0.861, 0.867]	[0.918, 0.923]
0.9-1.0 (time)	[0.359, 0.364]	[0.451, 0.457]	[0.596, 0.601]	[0.600, 0.605]	[0.733, 0.739]	[0.889, 0.894]	[0.922, 0.927]
0.95 [14]	[0.315, 0.321]	[0.400, 0.405]	[0.556, 0.562]	[0.558, 0.563]	[0.714, 0.720]	[0.866, 0.871]	[0.920, 0.926]
0.99 [14]	[0.210, 0.216]	[0.252, 0.257]	[0.411, 0.417]	[0.450, 0.455]	[0.633, 0.638]	[0.817, 0.823]	[0.896, 0.902]
1.0 [17]	[0.087, 0.093]	[0.119, 0.124]	[0.247, 0.252]	[0.291, 0.296]	[0.525, 0.530]	[0.759, 0.764]	[0.879, 0.885]

Table 8. 95% confidence interval of *F-measure*

	Normalized utilization						
Threshold	275%	280%	285%	290%	295%	300%	305%
0.7-1.0 (job)	[0.507, 0.513]	[0.599, 0.605]	[0.738, 0.743]	[0.736, 0.741]	[0.849, 0.855]	[0.939, 0.945]	[0.955, 0.960]
0.7-1.0 (time)	[0.629, 0.635]	[0.669, 0.675]	[0.761, 0.766]	[0.774, 0.779]	[0.858, 0.864]	[0.942, 0.948]	[0.958, 0.963]
0.8-1.0 (job)	[0.492, 0.497]	[0.574, 0.580]	[0.733, 0.738]	[0.723, 0.728]	[0.841, 0.847]	[0.937, 0.942]	[0.953, 0.958]
0.8-1.0 (time)	[0.621, 0.627]	[0.637, 0.643]	[0.759, 0.765]	[0.781, 0.786]	[0.853, 0.858]	[0.941, 0.947]	[0.959, 0.965]
0.9-1.0 (job)	[0.457, 0.463]	[0.535, 0.540]	[0.701, 0.706]	[0.710, 0.716]	[0.829, 0.835]	[0.926, 0.931]	[0.950, 0.956]
0.9-1.0 (time)	[0.529, 0.535]	[0.616, 0.621]	[0.742, 0.747]	[0.745, 0.750]	[0.841, 0.846]	[0.937, 0.942]	[0.953, 0.959]
0.95 [14]	[0.477, 0.483]	[0.566, 0.571]	[0.716, 0.721]	[0.713, 0.719]	[0.832, 0.838]	[0.923, 0.929]	[0.954, 0.960]
0.99 [14]	[0.350, 0.355]	[0.402, 0.407]	[0.580, 0.586]	[0.620, 0.625]	[0.773, 0.778]	[0.897, 0.902]	[0.942, 0.947]
1.0 [17]	[0.167, 0.173]	[0.213, 0.219]	[0.396, 0.401]	[0.449, 0.455]	[0.689, 0.694]	[0.862, 0.868]	[0.934, 0.939]

Just Accepted

Furthermore, Welch's t-test was conducted to assess the statistical significance of the differences between the proposed methods and the baselines. The analysis confirmed that the differences in mean values for *Accuracy*, *Precision*, and *F-measure* between the proposed workload-based thresholding and the baselines are statistically significant, with p-values satisfying $p < 0.05$ (and frequently $p \ll 0.001$) across the evaluated utilization levels. Detailed tabulation of individual p-values is omitted for brevity, given the consistency of the results across the extensive set of test cases.

Table 9. *Earlier time* results

Probability threshold	Average *earlier time* [ms]
0.7-1.0 (job)	137.3
0.7-1.0 (time)	133.3
0.8-1.0 (job)	145.6
0.8-1.0 (time)	140.7
0.9-1.0 (job)	162.1
0.9-1.0 (time)	155.1
0.95	162.7
0.99	197.7
1.00	286.1

Regarding *Recall*, p-values exceeded 0.05 in specific instances. This is attributed to the ceiling effect, where both the proposed and baseline methods achieve a *Recall* of nearly 1.0. This indicates that the proposed method succeeds in reducing pessimism (improving *Precision*) without statistically compromising the safety (*Recall*) of the baseline.

5.4 Early Detection of Potential Deadline Misses

The ability of the proposed method to predict deadline misses in advance is evaluated in this section. The term *earlier time* refers to how much earlier a deadline miss can be identified compared to conventional methods. By adopting the proposed method, systems can anticipate a deadline miss and transition to a safe state significantly earlier compared to scenarios where no early detection mechanism is implemented. The task sets and parameters utilized for this evaluation are consistent with those outlined in Section 5.3 and Table 4.

The results for *earlier time* at various probability thresholds are summarized in Table 9. As mentioned previously, a higher threshold enforces stricter deadline miss predictions, whereas a lower threshold leads to more lenient predictions. Generally, higher thresholds result in earlier detection of deadline misses, as illustrated in Table 9. However, it is important to note that a larger *earlier time* value does not necessarily indicate a superior method. This is because predictions made significantly earlier might lack precision, resulting in a system that is overly pessimistic. For example, overly early predictions might lead to frequent transitions to safe states, even when no actual deadline misses occur. In such cases, most of the predicted deadline misses might not materialize, leading to an overly pessimistic system. Thus, comparing *earlier time* values across thresholds in isolation may not yield meaningful insights. Instead, evaluating *earlier time* alongside the accuracy of deadline miss detection at each threshold is more informative. As previously mentioned, setting the probability threshold to 0.95 significantly improves accuracy compared to a threshold of 1.00 (i.e., using a fixed WCET). In particular, *Precision* in scenarios with low deadline miss rates improves by approximately threefold. At this point, the *earlier time* is 162.7 ms, which remains meaningful in dynamic systems such as autonomous driving, where conditions change rapidly. When the threshold varies between 0.8 and 1.0 based on the total average execution time of remaining jobs, *Precision* in low deadline miss rate scenarios further improves by approximately 1.5 times compared to a global probability threshold of 0.95. In this case, the *earlier time* is 140.7 ms, approximately 90% of the value observed under a threshold of 0.95. Adjusting the probability threshold according to the remaining workload improves accuracy while maintaining *earlier time*. Given that real-world systems require sufficiently low deadline miss rates, the improvement in *Precision* has a significant impact.

Just Accepted

Table 10. Experimental parameters in Section 5.5

Parameter	Value
α in Equation (3)	2.0
Period of *timer-driven nodes* [ms]	10, 20, 30, 50, 60, 100

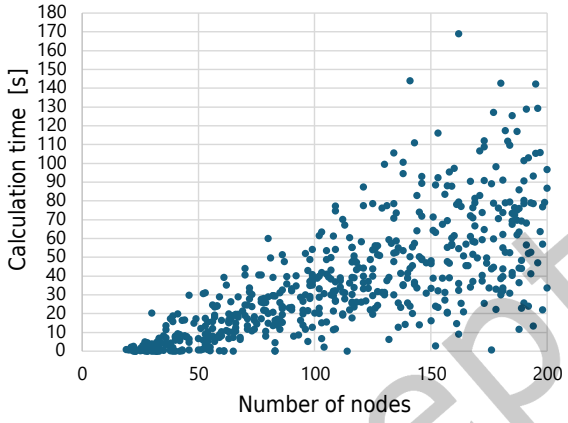


Fig. 10. Calculation time of *plaxity* and thresholds with increasing nodes.

5.5 Calculation Time for *plaxity* and the Threshold of Each Job

The computational cost of deriving *plaxity* and calculating probability threshold was evaluated with respect to two factors: (i) the number of nodes in the system, and (ii) the granularity of execution time units. A smaller execution time unit increases the number of entries in the execution time distribution for each task, which impacts the calculation complexity. The parameter settings common to the experiments for these two factors are detailed in Table 10.

First, the relationship between the number of nodes in a system and the calculation time for *plaxity* was analyzed. For this experiment, 500 DAGs were generated with the number of nodes randomly set in the range $[20, 200]$. The execution time unit was fixed at 10^{-4} seconds. The results, shown in Fig. 10, indicate that the calculation time generally increases with the number of nodes. Despite this trend, significant variations in calculation time were observed for DAGs containing a large number of nodes. This variation arises because the total number of *plaxity* calculations is not solely determined by the number of nodes. *plaxity* is computed only for jobs that have direct or indirect dependencies on exit jobs. The number of such jobs depends on the structure of the DAG and the differences in periods among subgraphs exchanging data. Nonetheless, larger DAGs tend to include more jobs requiring *plaxity* calculations, resulting in a correlation between the number of nodes and calculation time.

Focusing on the case where the number of nodes is 100, which approximates the typical size of DAGs in *Autoware* [6], the maximum calculation time is about a minute. Since *plaxity* calculations are performed statically during system development, this duration is well within a practical range. Even for DAGs with 200 nodes, the calculation time remains around 150 seconds, confirming the feasibility of the method for systems with a large number of nodes.

Just Accepted

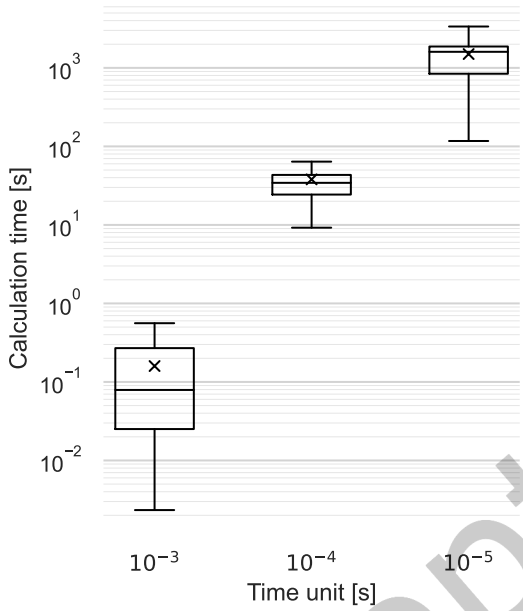


Fig. 11. Calculation time of *plaxity* and thresholds for different time units.

Next, the relationship between the granularity of execution time units and the calculation time for *plaxity* was analyzed. In this experiment, execution time units were set to 10^{-3} seconds, 10^{-4} seconds, and 10^{-5} seconds, and the calculation time was measured for 100 DAGs. For example, the execution time distribution of a task with a WCET of 5 ms contains 5, 50, and 500 entries under these respective units. When the unit is 10^{-3} seconds, all probabilities for execution times greater than 4 ms but less than 5 ms are grouped into the probability for 5 ms, introducing pessimism. While finer time units improve accuracy, they significantly increase the computational cost and warrant detailed investigation. For this experiment, the number of nodes in each DAG was fixed at 100 to isolate the effect of time unit granularity. The results, shown in Fig. 11, indicate that at 10^{-3} seconds, the calculation time is extremely short and poses no practical concerns. At 10^{-4} seconds, as noted earlier, the calculation time remains at a practical level. However, at 10^{-5} seconds, the maximum calculation time is under one hour. Although this is longer than for coarser units, it remains feasible considering that *plaxity* is computed statically.

To provide a theoretical foundation for the scalability observed in our experiments, we formalize the worst-case time complexity of the proposed algorithm. Let $|E|$ denote the number of dependency edges in the job-level DAG, and L denote the number of discrete intervals (bins) in the probabilistic execution time distribution, which depends on the time unit granularity.

The computational cost is dominated by the *plaxity* calculation, which involves convolution operations recursively from the exit jobs. Although *plaxity* is computed only for jobs that have direct or indirect dependencies on exit jobs, in the worst-case scenario (where all jobs are part of the dependency chain), the operation is performed for every edge. The standard discrete convolution of two arrays of size L has a time complexity of $O(L^2)$. Thus, the baseline complexity would be $O(|E| \cdot L^2)$. However, as described in Section 1, this paper implements circular convolution [11] to enhance efficiency. The complexity of circular convolution is reduced to $O(L \log L)$. Consequently, the theoretical worst-case time complexity of our *plaxity* calculation is $O(|E| \cdot L \log L)$.

Just Accepted

Table 11. Comparison of the proposed method with related work

	DAG	**MPD**[a]	**MTE**[b]	**DED**[c]	**PRB**[d]	**WPT**[e]
RTCSA 2016 [2]	✓	✓				
RTAS 2019 [15]	✓	✓				
ECRTS 2019 [4]	✓	✓	✓			
IEEE Access 2023 [17]	✓	✓	✓	✓		
RTSS 2002 [3]					✓	
IEEE Transactions on Computers 2022 [9]	✓	✓			✓	
RTAS 2023 [5]	✓	✓	✓		✓	
ECRTS 2024 [14]	✓	✓	✓	✓	✓	
Proposed method	✓	✓	✓	✓	✓	✓

[a] MPD: Multi-period DAG
[b] MTE: Mixture of *timer-driven* and *event-driven nodes*
[c] DED: Deadline miss early detection
[d] PRB: Probabilistic method
[e] WPT: Workload-aware probability threshold

Just Accepted

The threshold calculation requires a graph traversal with simple arithmetic operations, resulting in a complexity of $O(|N| + |E|)$, where $|N|$ is the number of jobs. This is negligible compared to the *plaxity* calculation.

Therefore, the overall worst-case complexity is dominated by $O(|E| \cdot L \log L)$. This linear dependence on the number of edges $|E|$ is particularly important for scalability. Since real-world autonomous driving systems, such as *Autoware*, typically consist of hundreds of nodes [6], this complexity class ensures that our method remains computationally feasible even for such large-scale systems, as supported by the observed calculation times.

Overall, the total time required for calculating *plaxity* and the probability threshold is considered to be within a practical limits. In this evaluation, circular convolution was employed for the convolution process, but its effectiveness is limited. This limitation likely arises from the relatively small size of the matrices subject to convolution, where the overhead outweighs the benefits, resulting in only a marginal advantage over linear convolution. However, using a finer granularity of execution times may lead to a more gradual increase in computation time as the system scales.

While our evaluation relies on statistical models derived from empirical data to construct the execution-time distributions, accurately modeling the tail probabilities is challenging in real-world systems. Minor deviations in tail probability estimation can affect the derived *plaxity* due to the nature of convolution. Specifically, underestimating the tail probability leads to an optimistic *plaxity*, potentially delaying the detection of actual deadline misses (increasing False Negatives). Conversely, overestimating the tail probability introduces pessimism, which could increase False Positives. However, the proposed workload-aware variable threshold mechanism mitigates this sensitivity by design. By relaxing thresholds during the early stages of the DAG and progressively tightening them, the system absorbs minor estimation inaccuracies without immediately triggering unnecessary fail-safe transitions.

6 Related Work

This section provides an overview of research related to this paper. A comparative summary of the reviewed approaches is presented in Table 11.

Approaches to identifying job-level dependencies in high-priority (HP) scenarios have been explored to facilitate timing analysis of DAGs comprising tasks driven by different periods. One heuristic solution was introduced by Becker et al. [2]. Their technique examines the earliest and latest times for data reads and writes associated with each periodic task to derive all possible job-level dependencies. Additionally, Verucchi et al. [15] proposed an alternative method that addresses this problem by converting a multi-period DAG into multiple single-period DAGs, thereby simplifying the determination of job-level dependencies in HP. While these methods effectively resolve dependencies in multi-period DAGs, their focus on exclusively periodic tasks limits their applicability to systems that integrate both *timer-driven* and *event-driven nodes*.

Systems combining *timer-driven* and *event-driven nodes* have been examined extensively within the context of robot operating systems (ROS). For example, Casini et al. [4] analyzed the response times of processing chains in ROS 2 and proposed scheduling techniques suitable for real-time operations. However, their approach does not address job-level dependencies, leaving this gap unaddressed in ROS-based systems.

Building on these advancements, Yano et al. [17] proposed a method for early detection of deadline misses in DAG tasks incorporating both *timer-driven* and *event-driven nodes*. Their method initially identifies job-level dependencies in such mixed DAGs, subsequently imposing time constraints on individual jobs based on the end-to-end deadlines and *data freshness* requirements. At runtime, the system evaluates whether the start time of a job complies with the predetermined time constraints to facilitate early detection of deadline misses. Nonetheless, this method inherits pessimism from its reliance on WCET assumptions when calculating time constraints.

Addressing pessimism stemming from WCET assumptions remains a recurring challenge in real-time systems. Probabilistic modeling has emerged as a promising approach to mitigate this issue, enabling a more nuanced analysis of system timing behavior. Bernat et al. [3] pioneered a framework using the convolution of probability mass functions to represent the execution times of program components. By iteratively applying convolutional operations, they derived the total execution time of a program as a probability mass function. Lee et al. [9] extended this methodology to autonomous driving systems, introducing probabilistic analyses for the end-to-end latency of multi-period DAGs within such systems. Further enhancing these techniques, Han et al. [5] proposed methods to reduce end-to-end latency by leveraging probabilistic execution time to counteract pessimistic estimations. Although they utilize probabilistic execution times, their focus is on minimizing end-to-end latency through scheduling rather than runtime deadline miss prediction. Consequently, a direct quantitative comparison with these methods is not conducted in this study.

While machine learning (ML) and reinforcement learning (RL) based approaches have been explored for WCET estimation or scheduling optimization in periodic task systems, they are not directly applicable to the complex mixed *timer-driven* and *event-driven* DAG models addressed in this paper without significant architectural modifications. Furthermore, given the safety-critical nature of autonomous driving, our approach prioritizes interpretability and computational efficiency on embedded platforms over the high inference costs often associated with online ML/RL methods.

7 Conclusion

This paper presented an enhanced method for early detection of deadline misses in real-time systems, incorporating variations in task execution times. The proposed method treats the execution time of each task as a random variable and uses convolution operations to define intermediate time constraints, referred to as *plaxity*, for every job within the DAG. These constraints facilitates early detection of deadline misses while enabling users to adjust the detection sensitivity by selecting appropriate probability thresholds. By accounting for execution time variability, the proposed method mitigates the pessimism inherent in conventional WCET-based approaches. Additionally, the method adjusts probability thresholds based on the remaining workload, using either uniform job contributions or average execution times of remaining jobs. Experimental results demonstrated that

Just Accepted

this workload-aware adjustment improves the balance between false positives and missed detections, reducing unnecessary system interruptions without compromising the detection accuracy.

Future work will focus on extending this approach to incorporate execution time variability into job-level dependency analysis. In particular, the RST (Reference Start Time) and RFT (Reference Finish Time) values that are used in our algorithms are currently calculated via the equations derived in Ref. [17], which rely on WCET assumptions; incorporating probabilistic models of the execution time of tasks into the computation of RST and RFT may further enhance detection accuracy. Future work will also explore the use of machine learning techniques for the offline optimization of probability thresholds to further adapt to varying workload conditions without incurring runtime computational overhead. Furthermore, we intend to investigate lightweight online monitoring mechanisms that allow for dynamic threshold adjustment or distribution updates at runtime, reducing the reliance on static tuning while maintaining computational feasibility.

References

[1] Baidu, Inc. 2024. *Baidu Apollo project.* Retrieved December 13, 2024 from https://www.apollo.auto/

[2] Matthias Becker, Dakshina Dasari, Saad Mubeen, Moris Behnam, and Thomas Nolte. 2016. Synthesizing Job-Level Dependencies for Automotive Multi-rate Effect Chains. In *Proceedings of the 22nd IEEE International Conference on Embedded and Real-Time Computing Systems and Applications (RTCSA)*. 159–169.

[3] Guillem Bernat, Antoine Colin, and Stefan M. Petters. 2002. WCET analysis of probabilistic hard real-time systems. In *Proceedings of the 23rd IEEE Real-Time Systems Symposium (RTSS)*. 279–288.

[4] Daniel Casini, Tobias Blaß, Ingo Lütkebohle, and Björn B. Brandenburg. 2019. Response-Time Analysis of ROS 2 Processing Chains Under Reservation-Based Scheduling. In *Proceedings of the 21st Euromicro Conference on Real-Time Systems (ECRTS)*. 1–23.

[5] Taeho Han and Kanghee Kim. 2023. Minimizing Probabilistic End-to-end Latencies of Autonomous Driving Systems. In *Proceedings of the 29th IEEE Real-Time and Embedded Technology and Applications Symposium (RTAS)*. 27–39.

[6] Shinpei Kato, Shota Tokunaga, Yuya Maruyama, Seiya Maeda, Manato Hirabayashi, Yuki Kitsukawa, Abraham Monrroy, Tomohito Ando, Yusuke Fujii, and Takuya Azumi. 2018. Autoware on Board: Enabling Autonomous Vehicles with Embedded Systems. In *Proceedings of the 9th ACM/IEEE International Conference on Cyber-Physical Systems (ICCPS)*. 287–296.

[7] Alix Munier Kordon and Ning Tang. 2020. Evaluation of the age latency of a real-time communicating system using the LET paradigm. In *Proceedings of the 32nd Euromicro Conference on Real-Time Systems (ECRTS)*. 1–21.

[8] Takahisa Kuboichi, Atsushi Hasegawa, Bo Peng, Keita Miura, Kenji Funaoka, Shinpei Kato, and Takuya Azumi. 2022. CARET: Chain-Aware ROS 2 Evaluation Tool. In *Proceedings of the 20th IEEE International Conference on Embedded and Ubiquitous Computing (EUC)*. 1–8.

[9] Hyoeun Lee, Youngjoon Choi, Taeho Han, and Kanghee Kim. 2022. Probabilistically Guaranteeing End-to-End Latencies in Autonomous Vehicle Computing Systems. *IEEE Trans. Comput.* 71, 12 (2022), 3361–3374.

[10] Jing Li, Jian Jia Chen, Kunal Agrawal, Chenyang Lu, Chris Gill, and Abusayeed Saifullah. 2014. Analysis of Federated and Global Scheduling for Parallel Real-Time Tasks. In *Proceedings of the 26th Euromicro Conference on Real-Time Systems (ECRTS)*. 85–96.

[11] Filip Marković, Alessandro Vittorio Papadopoulos, and Thomas Nolte. 2021. On the Convolution Efficiency for Probabilistic Analysis of Real-Time Systems. In *Proceedings of the 33rd Euromicro Conference on Real-Time Systems (ECRTS)*. 16:1–16:22.

[12] SAE International 2021. *Taxonomy and Definitions for Terms Related to Driving Automation Systems for On-Road Motor Vehicles* (4th ed.). SAE International.

[13] John A Stankovic, Marco Spuri, Krithi Ramamritham, and Giorgio Buttazzo. 1998. *Deadline scheduling for real-time systems: EDF and related algorithms*. Vol. 460. Springer Science & Business Media.

[14] Hayate Toba and Takuya Azumi. 2024. Deadline Miss Early Detection Method for DAG Tasks Considering Variable Execution Time. In *Proceedings of the 36th Euromicro Conference on Real-Time Systems (ECRTS)*, Vol. 298. 8:1–8:21.

[15] Micaela Verucchi, Mirco Theile, Marco Caccamo, and Marko Bertogna. 2020. Latency-Aware Generation of Single-Rate DAGs from Multi-Rate Task Sets. In *Proceedings of the 26th IEEE Real-Time and Embedded Technology and Applications Symposium (RTAS)*. 226–238.

[16] Waymo LLC 2024. *Waymo driver.* Retrieved December 13, 2024 from https://waymo.com/waymo-driver/

[17] Atsushi Yano and Takuya Azumi. 2023. Deadline Miss Early Detection Method for Mixed Timer-Driven and Event-Driven DAG Tasks. *IEEE Access* 11 (2023), 22187–22200.

[18] Atsushi Yano and Takuya Azumi. 2023. RD-Gen: Random DAG Generator Considering Multi-rate Applications for Reproducible Scheduling Evaluation. In *Proceedings of the 26th IEEE International Symposium on Real-Time Distributed Computing (ISORC)*. 21–31.

Received 31 January 2025; revised 29 December 2025; accepted 30 March 2026

Just Accepted